



南方科技大学

LAB 1: Python Review

COE 201 - AI Large Models, Spring 2026

2026-05-17

College of Engineering, SUSTech

Office Hour: Thursday, 14:30-15:30 @ 3rd Research Building, Room 610

Outline

- Lab Courses Intro 1
- Python Review 4
- Lab Tasks 33

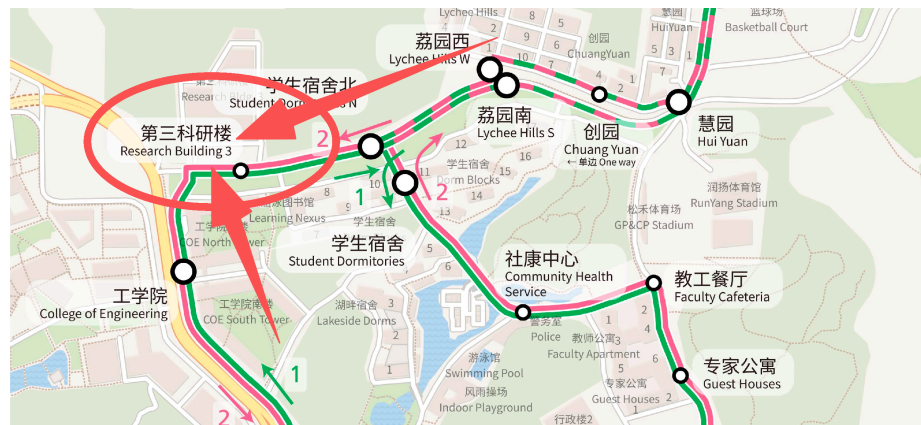
Lab Courses Intro

About COE 201



南方科技大学

- COE201 topic: **Large Model** from scratch
- About me: Peng Weiyuan (彭维源), RA @ HQ Lab
- Office hour: Thur, 14:30 - 15:30 (except for public holidays)
 - Find me @ 3rd Research Building (三科), Room 610



- No separate QQ group for the lab course (use the main course group)

Since COE201 is a second-year course, we won't make the labs overly difficult, so take it easy.

LLM usage is allowed, but follow this order when you get stuck:

1. **Try it yourself first** - spend some time debugging and thinking
2. **Ask the TA or classmates** - discuss the problem, we're here to help!
3. **Then use LLM** - if you still need assistance

This helps you actually learn rather than just getting answers.

Python Review

Why Python



南方科技大学

- Widely used, general purpose
- Easy to learn, read, and write
- Used by major deep learning and LLM frameworks (PyTorch, Transformers)
- Active open-source, many libraries!

Jan 2026	Jan 2025	Change	Programming Language	Ratings	Change
1	1		 Python	22.61%	-0.68%
2	4	▲	 C	10.99%	+2.13%
3	3		 Java	8.71%	-1.44%
4	2	▼	 C++	8.67%	-1.62%
5	5		 C#	7.39%	+2.94%
6	6		 JavaScript	3.03%	-1.17%
7	9	▲	 Visual Basic	2.41%	+0.04%
8	8		 SQL	2.27%	-0.14%
9	11	▲	 Delphi/Object Pascal	1.98%	+0.19%
10	18	▲	 R	1.82%	+0.81%

TIOBE Index for January 2026

Why Environment Management?



Problem

- Different versions of Python, e.g. 3.11, 3.9 ...
- Countless Python packages and their dependencies
- Different projects require different packages

Even worse, different versions of the same package!

Solution

- Keep multiple Python environments that are isolated from each other
- Each environment
 1. Can use different Python version
 2. Keeps its own set of packages (can specify package versions)

We don't want to spend excessive time on environment setup, **but this is an important skill for LLM development.**

`uv` is a modern solution for Python environment management. Install from <https://docs.astral.sh/uv/getting-started/installation>.

Basic Workflow:

- Install Python: `uv python install python3.12`
- Initialize new `pyproject.toml`: `uv init`
- Add packages: `uv add 'numpy>2.0'`
- Use existing `pyproject.toml`: `uv sync`

- PyCharm
- Visual Studio Code
- Vim (for Linux or Mac) ...

Write a Python program in your IDE or text editor of choice.

Run a Python program in terminal:

```
1 # Activate virtual env (venv) and run program with command
2 source .venv/bin/activate # if using bash-compatible shell
3 python <filename.py>
4 deactivate # deactivate current venv
5 # or use `uv run` directly (do not need venv activated):
6 uv run <filename.py>
```



Basic Data Types: Numbers



Integers and floats work as you would expect from other languages:

```
1  # Integer operations
2  x = 3
3  print(x)                # OUTPUT: 3
4  print(x + 1, x - 1)      # OUTPUT: 4 (addition) 2 (subtraction)
5  print(x * 2, x ** 2)     # OUTPUT: 6 (multiplication) 9 (exponentiation)
6  # Augmented assignment
7  x += 1                   # x becomes 4
8  print(x)                 # OUTPUT: 4
9  x *= 2                   # x becomes 8
10 print(x)                 # OUTPUT: 8
11 # Float operations
12 y = 2.5
13 print(y, y + 1, y * 2, y ** 2) # OUTPUT: 2.5 3.5 5.0 6.25
```

 Python

Basic Data Types: Strings



```
1  # Basic operations
2  s1, s2 = "hello", "world"
3  print(s1 + " " + s2)          # OUTPUT: hello world (concatenation)
4  print(s1 * 3)                 # OUTPUT: hellohellohello (repetition)
5
6  # Indexing and slicing
7  s = "hello world"
8  print(s[0])                   # OUTPUT: h (first character)
9  print(s[-1])                  # OUTPUT: d (last character)
10 print(s[0:5])                  # OUTPUT: hello (slicing)
11 print(s[::-1])                 # OUTPUT: dlrow olleh (reverse)
12
13 # Common methods
14 text = "  Hello, World!  "
15 print(text.strip())            # OUTPUT: Hello, World! (remove whitespace)
16 print(text.replace("World", "Python")) # OUTPUT:  Hello, Python! (replace)
17 print(text.split(","))         # OUTPUT: ['  Hello', ' World!  '] (split into list)
18
```

 Python

Basic Data Types: Strings



```
19 # Strings are immutable - this would cause TypeError
20 # text[0] = "H"                # TypeError: 'str' object does not support item assignment
21
22 # F-strings (formatted string literals)
23 name, age = "Alice", 30
24 print(f"Name: {name}, Age: {age}")      # OUTPUT: Name: Alice, Age: 30
25 print(f"Next year: {age + 1}")          # OUTPUT: Next year: 31
26 pi = 3.1415926
27 print(f"PI to 3 decimals: {pi:.3f}")    # OUTPUT: PI to 3 decimals: 3.142
```

`list` is appendable and can contain elements of different types:

```
1  # Creating and accessing lists
2  xs = [3, 1, 2]          # Create a list
3  print(xs)              # OUTPUT: [3, 1, 2]
4  print(xs[2])           # OUTPUT: 2 (access by index)
5  print(xs[-1])          # OUTPUT: 2 (negative index from end)
6
7  # Lists are mutable
8  xs[2] = 'foo'          # Modify element
9  print(xs)              # OUTPUT: [3, 1, 'foo']
10
11 # List methods
12 xs.append('bar')        # Add element to end
13 print(xs)              # OUTPUT: [3, 1, 'foo', 'bar']
14 # Loop over a list
15 xs = [3, 1, 'foo', 'bar']
16 for x in xs:
```

 Python

```
17 print(x)                # OUTPUT: 3 (then 1, 'foo', 'bar' on separate lines)
18
19 # Slicing: list elements can be accessed in convenient ways
20 # Basic format: some_list[start_index:end_index]
21 numbers = [0, 1, 2, 3, 4, 5, 6]
22 numbers[0:3] == numbers[:3] == [0, 1, 2]
23 numbers[5:] == numbers[5:7] == [5, 6]
24 numbers[:] == numbers == [0, 1, 2, 3, 4, 5, 6]
25 numbers[-1] == 6 # Negative index wraps around
26 numbers[-3:] == [4, 5, 6]
27 numbers[3:-2] == [3, 4] # Can mix and match
```

Data Container: Tuples



`tuple` is immutable and can contain elements of different types:

```
1  # Creating tuples
2  t = (1, 2, 3)          # Parentheses recommended
3  single = (1,)         # Single-element tuple (note comma)
4  empty = ()            # Empty tuple
5
6  # Indexing and slicing
7  print(t[0])           # OUTPUT: 1
8  print(t[1:3])         # OUTPUT: (2, 3) (slicing)
9  print(len(t))         # OUTPUT: 3
10
11 # Tuples are immutable
12 # t[0] = 10           # TypeError: 'tuple' object does not support item assignment
13
14 # Tuple unpacking
15 x, y, z = t           # Unpack tuple into variables
16 print(x, y, z)        # OUTPUT: 1 2 3
```



Data Container: Tuples



```
17
18 # Swapping variables using tuple unpacking
19 a, b = 10, 20
20 a, b = b, a          # Swap values
21 print(f"a={a}, b={b}") # OUTPUT: a=20, b=10
```

A dictionary stores (key, value) pairs:

```
1  # Creating and accessing dictionaries
2  d = {'cat': 'cute', 'dog': 'furry'}  # Create dictionary
3  print(d['cat'])                      # OUTPUT: cute (access by key)
4  print('cat' in d)                    # OUTPUT: True (check key existence)
5
6  # Adding and modifying entries
7  d['fish'] = 'wet'                    # Add new key-value pair
8  print(d['fish'])                     # OUTPUT: wet
9
10 # Safe access with get() method
11 # print(d['monkey'])                  # KeyError: 'monkey' not a key of d
12 print(d.get('monkey', 'N/A'))         # OUTPUT: N/A (default when key missing)
13 print(d.get('fish', 'N/A'))           # OUTPUT: wet (key exists)
14
15 # Removing entries
16 del d['fish']                         # Delete key-value pair
```

 Python

```
17 print(d.get('fish', 'N/A'))          # OUTPUT: N/A (key no longer exists)
18
19 # Dict keys must be hashable (immutable types like str, int, tuple)
20 # d[[1, 2]] = "list"      # TypeError: unhashable type: 'list'
21 d[(1, 2)] = "tuple works!" # Tuples are hashable
```

Functions are defined using the `def` keyword. For example:

```
1  # Simple function with default argument
2  def greet(name, greeting="Hello"):
3      return f"{greeting}, {name}!"
4
5  print(greet("Alice"))          # OUTPUT: Hello, Alice!
6  print(greet("Bob", "Hi"))      # OUTPUT: Hi, Bob!
7
8  # Recursive function (calls itself)
9  def factorial(n):
10     if n == 0:
11         return 1
12     return n * factorial(n - 1)
13 print(factorial(5))            # OUTPUT: 120 (5! = 5*4*3*2*1)
```

 Python

Basic class definition:

```
1  # Simple class with constructor and method
2  class Person:
3      def __init__(self, name, age):
4          self.name = name    # Instance variable
5          self.age = age      # Instance variable
6
7      def greet(self):
8          return f"Hello, my name is {self.name}"
9
10 # Create instance and call method
11 p = Person(name="Alice", age=30)
12 print(p.greet())           # OUTPUT: Hello, my name is Alice
```

 Python

Basic file operations in Python. Common open modes:

- "r" - Read (default), file must exist
- "w" - Write, creates new file or **overwrites** existing
- "a" - Append, adds to end of file
- "r+" - Read and write

```
1  # Writing to a file
2  with open("example.txt", "w") as f:
3      f.write("Hello, World!\n")
4
5  # Reading from a file
6  with open("example.txt", "r") as f:
7      content = f.read()
8      print(content)          # OUTPUT: Hello, World!
9
10 # Reading line by line
```

 Python

```
11 with open("example.txt", "r") as f:
12     for line in f:
13         print(f"Line: {line.strip()}") # OUTPUT: Line: Hello, World! etc.
14
15 # Appending to a file
16 with open("example.txt", "a") as f:
17     f.write("Appended line.\n")
```

Import Package Modules



Use keyword `import` to make use of package modules in your script:

```
1 # Import 'os' and 'time' standard library modules
2 import os, time
3
4 # Import under an alias
5 import numpy as np
6 np.dot(x, y) # Access components with pkg.fn
7 # Import specific submodules/functions
8 from numpy import linalg as la, dot as matrix_multiply
9 # Be careful: importing many names directly can cause conflicts
```



List Comprehensions



Similar to `map()` from functional programming languages (readability + succinct)

Format: `[func(x) for x in some_list]`

```
1 squares = []
2 for i in range(10):
3     squares.append(i**2)
4
5 # which is equivalent to
6 squares = [i**2 for i in range(10)]
```

 Python

Can use with `if`:

```
1 odds = [i**2 for i in range(10) if i%2 == 1]
```

 Python

Using pathlib.Path



Modern way to handle file paths (cross-platform, more readable than `os.path`):

```
1  from pathlib import Path
2
3  # Create path objects
4  data_dir = Path("data")
5  file_path = data_dir / "output" / "result.txt" # Use / to join paths
6
7  # Common operations
8  file_path.exists()      # Check if path exists
9  file_path.is_file()     # Check if it's a file
10 file_path.parent        # Get parent directory: data/output
11 file_path.name          # Get filename: result.txt
12 file_path.stem          # Get name without extension: result
13 file_path.suffix        # Get extension: .txt
14
15 # Read/write files directly
16 file_path.write_text("Hello!")
```



Using pathlib.Path



```
17 content = file_path.read_text()
18
19 # List files in directory
20 for f in Path(".").glob("*.py"):
21     print(f)           # All .py files in current dir
```

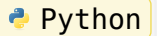
Debugging Tools: print & pdb



Debugging is a critical skill! Here are three approaches:

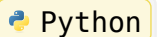
1. Print Debugging (Simple but effective)

```
1 print(f"DEBUG: x = {x}, type = {type(x)}") # Check variable values
2 print(f"DEBUG: reached line 42")           # Check code flow
```



2. pdb (Command-line debugger)

```
1 import pdb; pdb.set_trace() # Insert where you want to pause
2 # Common commands: n (next), s (step into), c (continue), p var (print)
```



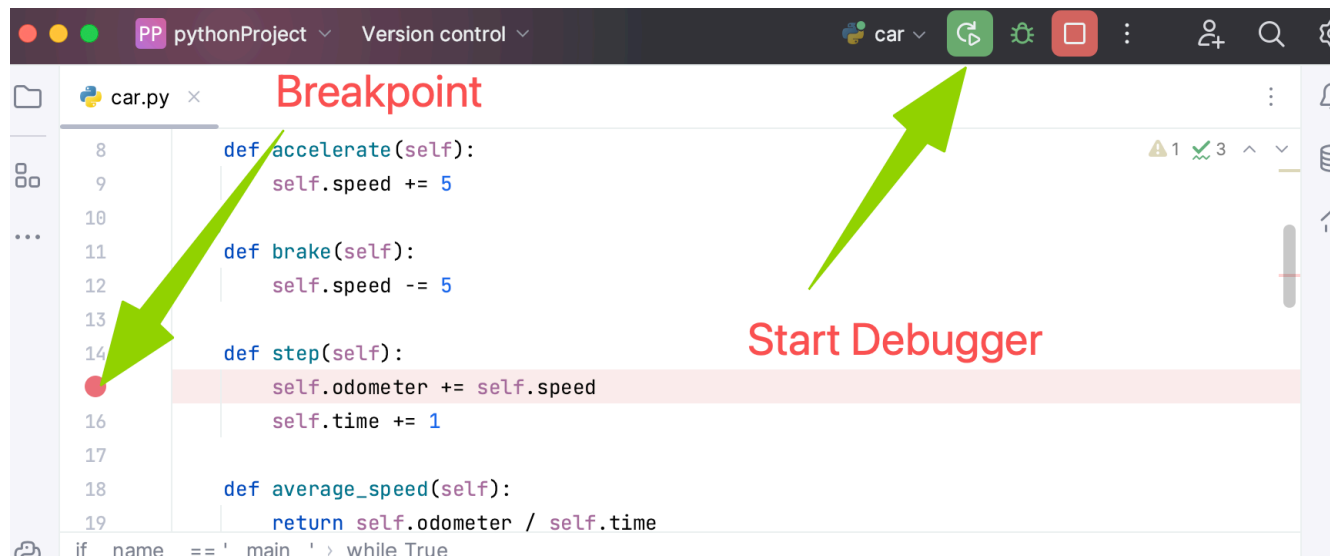
Docs: <https://docs.python.org/3.12/library/pdb.html>

Debugging Tools: IDE Debugger



南方科技大学

Set breakpoints, step through code, inspect variables. PyCharm example:



Other Great References



南方科技大学

1. Official Python 3 documentation: <https://docs.python.org/3/>
 - If you encounter any language/standard library problems
2. Python Official Tutorial: <https://docs.python.org/3/tutorial/>
3. uv “cheat sheet”: <https://docs.astral.sh/uv/getting-started/features/>
4. Python/Numpy tutorial: <https://cs231n.github.io/python-numpy-tutorial/>

Several Python and library-specific (ex. NumPy) “Cheat Sheet” guides online as well!

Why Vectorization?



南方科技大学

Deep learning models involve massive amounts of data and calculations, so **high performance is essential!**

Instead of using slow Python `for` loops, we use NumPy to perform operations on entire arrays at once.

Basic NumPy Operations



```
1  import numpy as np
2
3  # Create an array from a Python list
4  a = np.array([1, 2, 3])
5  print(a)          # OUTPUT: [1 2 3]
6
7  # Create a 2x2 matrix filled with zeros
8  z = np.zeros((2, 2))
9  print(z)          # OUTPUT: [[0. 0.] [0. 0.]]
10
11 # Mathematic operations are applied element-wise automatically
12 b = np.array([4, 5, 6])
13 print(a + b)       # OUTPUT: [5 7 9] (Addition)
14 print(a * 2)       # OUTPUT: [2 4 6] (Scalar multiplication)
15
16 # Dot product (Matrix multiplication)
17 print(np.dot(a, b))# OUTPUT: 32 (1*4 + 2*5 + 3*6)
18
```

 Python

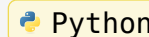
Basic NumPy Operations



```
19 # Common aggregation functions
20 c = np.array([-1.5, 2.0, 3.5])
21 print(np.sum(c))    # OUTPUT: 4.0 (Sum of all elements)
22 print(np.max(c))    # OUTPUT: 3.5 (Find the maximum value)
23 print(np.abs(c))    # OUTPUT: [1.5 2.  3.5] (Absolute value)
```

NumPy automatically aligns dimensions when performing operations on arrays of different shapes.

```
1  import numpy as np
2  A = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]]) # Shape: (3, 3)
3  b = np.array([1, 1, 1])                        # Shape: (3,)
4
5  # Broadcasting expands vector 'b' to match matrix 'A'
6  # Internally, 'b' is virtually duplicated to match A's shape:
7  # [[1,1,1],
8  #  [1,1,1],
9  #  [1,1,1]]
10 C = A + b
```



Lab Tasks

Complete the following tasks: **Task 0: Environment Check (`lab1_env_check.py`)**
[0 points]

- Install uv and sync environment.
- Run the check script to verify setup.

Task 1: Matrix Multiplication Benchmark (`lab1_benchmark.py`) [50 points]

- Experience the speedup of vectorization compared to nested loops.
- Measure performance difference between Python loops and NumPy operations.

Task 2: Vectorized Softmax (`lab1_math.py`) [50 points]

- Implement Softmax using NumPy broadcasting without any loops.
- Formula:



$$S(x_i) = \frac{e^{\{x_i\}}}{\sum (e^{\{x_j\}})}$$

Thanks for Listening!